

Guía Docente Diapositiva por Diapositiva — Sesión 6: Multi-Entorno Dev/Prod, Debugging Profesional y Despliegue Final

Curso: Docker desde Cero: Crea y Despliega Aplicaciones (10ma Edición 2026) **Instructor:** Ing. Cristian Jampier Chileno Segundo | OTI - UNI **Total Diapositivas:** 26

Instrucciones de Orientación Pedagógica

Esta guía contiene la explicación exacta y guión en primera persona para explicar **cada una de las diapositivas de la presentación oficial**. Úsala durante la clase virtual o presencial para garantizar que no quede ningún punto del temario sin abordar.

Explicación Diapositiva por Diapositiva

Diapositiva 1: DOCKER DESDE CERO:

Contenido de la PPT:

```
DOCKER DESDE CERO:  
Crea y Despliega Aplicaciones  
INSTRUCTOR: Cristian Jampier Chileno Segundo  
PROGRAMA DE  
INICIACIÓN  
TECNOLÓGICA  
PIT 2026  
Oficina de Tecnologías de la Información (OTI)  
Programa Completo ù PIT 2026
```

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 1** abordamos un punto fundamental: *DOCKER DESDE CERO*. Revisen los conceptos clave presentados en pantalla y asegúrense de que los estudiantes comprendan cómo se conecta este punto con el laboratorio práctico de la sesión."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 1  
./deploy.sh prod
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre DOCKER DESDE CERO?"

Diapositiva 2: SESIÖN 6

Contenido de la PPT:

SESIÖN 6

1. Configuraci³n Multi-Entorno
2. Manejo de .env por Entorno
3. Limpieza de Recursos
4. Debugging Profesional
5. Reconstrucci³n desde Cero
6. Despliegue e Integraci³n Final

Gui³n del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 2** abordamos un punto fundamental: *SESIÖN 6*. Enfoquen esta parte en las buenas pr³cticas de producci³n: separar **dev** (hot-reload, bind mounts) de **prod** (im³genes compiladas, Nginx, sin puertos directos a BD) y utilizar scripts automatizados **deploy.sh**."

 **Acci³n en Consola / Pizarra:** *Escribir en la pizarra o proyectar el diagrama del flujo correspondiente.*

Tip de gesti³n del Aula:

- *Hacer una pregunta de pausa activa: "¿Alguien tiene alguna duda sobre SESIÖN 6?"*

Diapositiva 3: Separar configuraci³n para entornos dev y prod.

Contenido de la PPT:

Separar configuraci³n para entornos dev y prod.
Ejecutar el proyecto final con archivos Compose especÝficos.
Limpiar contenedores, im³genes, redes y volúmenes huórfanos.
Diagnosticar fallas usando logs, inspect, exec y stats.
Destruir y reconstruir el stack completo con un flujo reproducible.
Validar que Flask, PostgreSQL y Nginx queden listos en minutos.
Al terminar la sesi³n 6 podr³s
Objetivo de la sesi³n 6

Gui³n del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 3** abordamos un punto fundamental: *Separar configuraci³n para entornos dev y prod.*. Resalten que en la vida real una aplicaci³n web necesita varios componentes:

app web, base de datos PostgreSQL, proxy Nginx. Docker Compose nos permite describir y levantar todo el stack con un único archivo `docker-compose.yml`."

 **Acción en Consola / Pizarra:** *Escribir en la pizarra o proyectar el diagrama del flujo correspondiente.*

 **Tip de Gestión del Aula:**

- *Hacer una pregunta de pausa activa: "¿Alguien tiene alguna duda sobre Separar configuración para entornos dev y prod?"*

 Diapositiva 4: Multi-entorno

Contenido de la PPT:

Multi-entorno

 **Guión del Docente (Lo que debes decir a los alumnos):**

"Estimados estudiantes, en la **Diapositiva 4** abordamos un punto fundamental: *Multi-entorno*. Enfoquen esta parte en las buenas prácticas de producción: separar `dev` (hot-reload, bind mounts) de `prod` (imágenes compiladas, Nginx, sin puertos directos a BD) y utilizar scripts automatizados `deploy.sh`."

 **Acción en Consola / Pizarra:** *Escribir en la pizarra o proyectar el diagrama del flujo correspondiente.*

 **Tip de Gestión del Aula:**

- *Hacer una pregunta de pausa activa: "¿Alguien tiene alguna duda sobre Multi-entorno?"*

 Diapositiva 5: En desarrollo necesitamos recarga rápida,

Contenido de la PPT:

En desarrollo necesitamos recarga rápida,
logs visibles y bind mounts.
En producción preferimos imagen
construida, menos puertos y configuración
cerrada.
El mismo proyecto debe poder levantarse
con reglas distintas según el entorno.
Compose permite combinar archivos para
reutilizar una base común.
No copies todo el YAML: define una base y agrega
diferencias por entorno.
Regla práctica
Por qué separar dev y prod

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 5** abordamos un punto fundamental: *En desarrollo necesitamos recarga rápida*. Resalten que en la vida real una aplicación web necesita varios componentes: app web, base de datos PostgreSQL, proxy Nginx. Docker Compose nos permite describir y levantar todo el stack con un único archivo `docker-compose.yml`."

 **Acción en Consola / Pizarra:** *Escribir en la pizarra o proyectar el diagrama del flujo correspondiente.*

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre En desarrollo necesitamos recarga rápida,?"

Diapositiva 6: ARCHIVOS DEL PROYECTO FINAL

Contenido de la PPT:

ARCHIVOS DEL PROYECTO FINAL

El archivo base describe la arquitectura; los archivos por entorno solo cambian lo necesario.

Idea

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 6** abordamos un punto fundamental: *ARCHIVOS DEL PROYECTO FINAL*. Revisen los conceptos clave presentados en pantalla y asegúrense de que los estudiantes comprendan cómo se conecta este punto con el laboratorio práctico de la sesión."

 **Acción en Consola / Pizarra:** *Escribir en la pizarra o proyectar el diagrama del flujo correspondiente.*

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre ARCHIVOS DEL PROYECTO FINAL?"

Diapositiva 7: COMPOSE.YML BASE

Contenido de la PPT:

COMPOSE.YML BASE

```
services:
web:
build: .
expose:
- "5000"
env_file:
- .env
```

```
depends_on:
db:
condition: service_healthy
db:
image: postgres:16
volumes:
- postgres_data:/var/lib/postgresql/data
nginx:
image: nginx:1.27-alpine
ports:
- "8080:80"
volumes:
postgres_data:
```

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 7** abordamos un punto fundamental: *COMPOSE.YML BASE*. Resalten que en la vida real una aplicación web necesita varios componentes: app web, base de datos PostgreSQL, proxy Nginx. Docker Compose nos permite describir y levantar todo el stack con un único archivo *docker-compose.yml*."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 7
./deploy.sh prod
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre COMPOSE.YML BASE?"

Diapositiva 8: Diferencias entre dev y prod

Contenido de la PPT:

```
Diferencias entre dev y prod
compose.dev.yml
compose.prod.yml
services:
web:
volumes:
- ./app:/app
environment:
FLASK_ENV: development
FLASK_DEBUG: "1"
services:
web:
image: usuario/flask-app:prod
restart: unless-stopped
```

```
environment:  
FLASK_ENV: production
```

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 8** abordamos un punto fundamental: *Diferencias entre dev y prod*. Resalten que en la vida real una aplicación web necesita varios componentes: app web, base de datos PostgreSQL, proxy Nginx. Docker Compose nos permite describir y levantar todo el stack con un único archivo `docker-compose.yml`."

 **Acción en Consola / Pizarra:** *Escribir en la pizarra o proyectar el diagrama del flujo correspondiente.*

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa: "¿Alguien tiene alguna duda sobre Diferencias entre dev y prod?"*

Diapositiva 9: COMANDOS POR ENTORNO

Contenido de la PPT:

```
COMANDOS POR ENTORNO  
# Desarrollo  
cp .env.dev .env  
docker compose \  
-f compose.yml \  
-f compose.dev.yml \  
up -d --build  
Ambos entornos deben responder desde http://localhost:8080, pero con configuraci%  
distinta.  
Validaci%  
# Produccion local  
cp .env.prod .env  
docker compose \  
-f compose.yml \  
-f compose.prod.yml \  
up -d
```

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 9** abordamos un punto fundamental: *COMANDOS POR ENTORNO*. Resalten que en la vida real una aplicación web necesita varios componentes: app web, base de datos PostgreSQL, proxy Nginx. Docker Compose nos permite describir y levantar todo el stack con un único archivo `docker-compose.yml`."

 **Acción en Consola / Pizarra:**

```
# Demostración práctica recomendada para la Diapositiva 9  
./deploy.sh prod
```

 Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre COMANDOS POR ENTORNO?"

 Diapositiva 10: Limpieza del**Contenido de la PPT:**

Limpieza del
proyecto

 Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 10** abordamos un punto fundamental: *Limpieza del*. Revisen los conceptos clave presentados en pantalla y asegúrense de que los estudiantes comprendan cómo se conecta este punto con el laboratorio práctico de la sesión."

 **Acción en Consola / Pizarra:** *Escribir en la pizarra o proyectar el diagrama del flujo correspondiente.*

 Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre Limpieza del?"

 Diapositiva 11: QUÉ SIGNIFICA LIMPIAR EN DOCKER**Contenido de la PPT:**

QUÉ SIGNIFICA LIMPIAR EN DOCKER
Limpiar no siempre es borrar todo. Los volúmenes pueden contener datos importantes.
Cuidado

 Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 11** abordamos un punto fundamental: *QUÉ SIGNIFICA LIMPIAR EN DOCKER*. Revisen los conceptos clave presentados en pantalla y asegúrense de que los estudiantes comprendan cómo se conecta este punto con el laboratorio práctico de la sesión."

 **Acción en Consola / Pizarra:**

```
# Demostración práctica recomendada para la Diapositiva 11
./deploy.sh prod
```

 Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre QUÉ SIGNIFICA LIMPIAR EN DOCKER?"
-

Diapositiva 12: LIMPIEZA SEGURA DEL STACK

Contenido de la PPT:

```
LIMPIEZA SEGURA DEL STACK
# Detener y eliminar contenedores/red
docker compose down
# Quitar contenedores huérfanos
docker compose down --remove-orphans
# Ver estado global
docker ps -a
docker images
docker volume ls
# Limpieza general segura
docker container prune
docker image prune
docker network prune
# Cuidado: borra volúmenes no usados
docker volume prune
```

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 12** abordamos un punto fundamental: *LIMPIEZA SEGURA DEL STACK*. Resalten que en la vida real una aplicación web necesita varios componentes: app web, base de datos PostgreSQL, proxy Nginx. Docker Compose nos permite describir y levantar todo el stack con un único archivo `docker-compose.yml`."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 12
./deploy.sh prod
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre LIMPIEZA SEGURA DEL STACK?"
-

Diapositiva 13: DESTRUIR TODO PARA RECONSTRUIR DESDE CERO

Contenido de la PPT:

```
DESTRUIR TODO PARA RECONSTRUIR DESDE CERO
# 1. Bajar el stack y borrar volúmenes del proyecto
docker compose down -v --remove-orphans
```



```
# 2. Eliminar imagen local del proyecto si existe
docker image rm mi-flask:prod || true
# 3. Reconstruir desde cero
docker compose -f compose.yml -f compose.prod.yml up -d --build
Este flujo debe dejar el proyecto funcional aunque la máquina empiece sin
contenedores
previos.
Regla de laboratorio final
```

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 13** abordamos un punto fundamental: *DESTRUIR TODO PARA RECONSTRUIR DESDE CERO*. Expliquen aquí el problema clásico en el desarrollo de software. En la laptop del desarrollador todo marcha perfecto porque tiene instaladas librerías específicas o variables locales, pero en el servidor de producción o en la PC del cliente falla. Docker soluciona esto empaquetando todo el entorno junto con la app."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 13
./deploy.sh prod
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre DESTRUIR TODO PARA RECONSTRUIR DESDE CERO?"

Diapositiva 14: Debugging

Contenido de la PPT:

Debugging
profesional

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 14** abordamos un punto fundamental: *Debugging*. Enfoquen esta parte en las buenas prácticas de producción: separar **dev** (hot-reload, bind mounts) de **prod** (imágenes compiladas, Nginx, sin puertos directos a BD) y utilizar scripts automatizados **deploy.sh**."

Acción en Consola / Pizarra: *Escribir en la pizarra o proyectar el diagrama del flujo correspondiente.*

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre Debugging?"

Diapositiva 15: ORDEN RECOMENDADO DE DIAGNÓSTICO

Contenido de la PPT:

ORDEN RECOMENDADO DE DIAGNÓSTICO

Primero mira qué está pasando; recién después cambia YAML, Dockerfile o código.
Regla práctica

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 15** abordamos un punto fundamental: *ORDEN RECOMENDADO DE DIAGNÓSTICO*. Expliquen que el Dockerfile es la receta de cocina. Cada instrucción (*FROM, WORKDIR, COPY, RUN, CMD*) construye una capa estandarizada e inmutable sobre la cual se monta nuestra aplicación."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 15
./deploy.sh prod
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre ORDEN RECOMENDADO DE DIAGNÓSTICO?"

Diapositiva 16: HERRAMIENTAS DE DEBUGGING

Contenido de la PPT:

HERRAMIENTAS DE DEBUGGING

COMANDO

PREGUNTA QUE RESPONDE

CUÁNDO USARLO

`docker compose ps`

`docker compose logs -f web`

`docker inspect <id>`

¿Qué servicio está arriba?

Primer chequeo

¿Qué error imprime la app?

¿Qué red, env y mounts tiene?

`docker compose exec web sh`

¿Qué pasa dentro del contenedor?

Fallas de arranque

Configuración dudosa

Pruebas manuales

`docker stats`

```
└─ Consume demasiados recursos?  
Rendimiento básico
```

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 16** abordamos un punto fundamental: *HERRAMIENTAS DE DEBUGGING*. Resalten que en la vida real una aplicación web necesita varios componentes: app web, base de datos PostgreSQL, proxy Nginx. Docker Compose nos permite describir y levantar todo el stack con un único archivo `docker-compose.yml`."

🏠 Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 16  
./deploy.sh prod
```

💡 Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre HERRAMIENTAS DE DEBUGGING?"

📄 Diapositiva 17: LABORATORIO: DIAGNOSTICAR EL STACK

Contenido de la PPT:

```
LABORATORIO: DIAGNOSTICAR EL STACK  
# Estado general  
docker compose ps  
# Logs por servicio  
docker compose logs -f nginx  
docker compose logs -f web  
docker compose logs -f db  
# Entrar a la app  
docker compose exec web sh  
# Probar desde dentro  
env | grep DB_  
curl http://localhost:5000  
# Recursos  
docker stats
```

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 17** abordamos un punto fundamental: *LABORATORIO: DIAGNOSTICAR EL STACK*. Resalten que en la vida real una aplicación web necesita varios componentes: app web, base de datos PostgreSQL, proxy Nginx. Docker Compose nos permite describir y levantar todo el stack con un único archivo `docker-compose.yml`."

🏠 Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 17
./deploy.sh prod
```

💡 Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre LABORATORIO: DIAGNOSTICAR EL STACK?"

📄 Diapositiva 18: Despliegue

Contenido de la PPT:

Despliegue
completo

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 18** abordamos un punto fundamental: *Despliegue*. Revisen los conceptos clave presentados en pantalla y asegúrense de que los estudiantes comprendan cómo se conecta este punto con el laboratorio práctico de la sesión."

🖨️ **Acción en Consola / Pizarra:** *Escribir en la pizarra o proyectar el diagrama del flujo correspondiente.*

💡 Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre Despliegue?"

📄 Diapositiva 19: ARQUITECTURA FINAL DEL CURSO

Contenido de la PPT:

ARQUITECTURA FINAL DEL CURSO
Primero mira qué está pasando; recién después cambia YAML, Dockerfile o código.
Resultado final

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 19** abordamos un punto fundamental: *ARQUITECTURA FINAL DEL CURSO*. Expliquen que el Dockerfile es la receta de cocina. Cada instrucción (**FROM**, **WORKDIR**, **COPY**, **RUN**, **CMD**) construye una capa estandarizada e inmutable sobre la cual se monta nuestra aplicación."

🖨️ **Acción en Consola / Pizarra:**

```
# Demostración práctica recomendada para la Diapositiva 19
./deploy.sh prod
```

💡 Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre ARQUITECTURA FINAL DEL CURSO?"

📄 Diapositiva 20: SCRIPT DE DESPLIEGUE FINAL

Contenido de la PPT:

```
SCRIPT DE DESPLIEGUE FINAL
#!/usr/bin/env bash
set -euo pipefail
cp .env.prod .env
docker compose \
-f compose.yml \
-f compose.prod.yml \
down -v --remove-orphans
docker compose \
-f compose.yml \
-f compose.prod.yml \
up -d --build docker compose ps
```

👤 Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 20** abordamos un punto fundamental: *SCRIPT DE DESPLIEGUE FINAL*. Resalten que en la vida real una aplicación web necesita varios componentes: app web, base de datos PostgreSQL, proxy Nginx. Docker Compose nos permite describir y levantar todo el stack con un único archivo `docker-compose.yml`."

🖥️ Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 20
./deploy.sh prod
```

💡 Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre SCRIPT DE DESPLIEGUE FINAL?"

📄 Diapositiva 21: VALIDACIÓN FINAL

Contenido de la PPT:

```
VALIDACIÖN FINAL
# 1. Ver servicios
docker compose ps
# 2. Probar endpoint
curl http://localhost:8080
# 3. Revisar logs
docker compose logs --tail=50 web
Nginx responde en puerto 8080.
Flask corre sin exponer puerto
directo.
PostgreSQL conserva datos en
volumen.
El stack se reconstruye sin pasos
manuales.
Debe cumplirse
```

Guiön del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 21** abordamos un punto fundamental: *VALIDACIÖN FINAL*. Resalten que en la vida real una aplicaciön web necesita varios componentes: app web, base de datos PostgreSQL, proxy Nginx. Docker Compose nos permite describir y levantar todo el stack con un ünico archivo *docker-compose.yml*."

Acciön en Consola / Pizarra:

```
# Demostraciön prätica recomendada para la Diapositiva 21
./deploy.sh prod
```

Tip de Gestiön del Aula:

- *Hacer una pregunta de pausa activa: "¿Alguien tiene alguna duda sobre VALIDACIÖN FINAL?"*

Diapositiva 22: ERRORES FRECUENTES EN EL PROYECTO FINAL

Contenido de la PPT:

```
ERRORES FRECUENTES EN EL PROYECTO FINAL
ERROR
CAUSA PROBABLE
SOLUCIÖN RÄPIDA
Usa configuraciön equivocada
Nginx no llega a Flask
BD inicia pero app falla
Se mezclä .env.dev con prod
Copiar el .env correcto
Servicio o puerto incorrecto
PostgreSQL no estä listo
```

```
Datos desaparecen
Se ejecut% down -v
Revisar proxy_pass y compose ps
Healthcheck y logs de db
Restaurar backup o evitar borrar volumen
Rebuild no cambia nada
CachÚ o imagen antigua
Usar up -d ûbuild y revisar tags
```

👤 **Guión del Docente (Lo que debes decir a los alumnos):**

"Estimados estudiantes, en la **Diapositiva 22** abordamos un punto fundamental: *ERRORES FRECUENTES EN EL PROYECTO FINAL*. Resalten que en la vida real una aplicación web necesita varios componentes: app web, base de datos PostgreSQL, proxy Nginx. Docker Compose nos permite describir y levantar todo el stack con un único archivo `docker-compose.yml`."

🏠 **Acción en Consola / Pizarra:**

```
# Demostración práctica recomendada para la Diapositiva 22
./deploy.sh prod
```

💡 **Tip de Gestión del Aula:**

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre ERRORES FRECUENTES EN EL PROYECTO FINAL?"

📄 Diapositiva 23: CHECKLIST DE APRENDIZAJE ù SESIËN 6

Contenido de la PPT:

```
CHECKLIST DE APRENDIZAJE ù SESIËN 6
1.Puedo explicar por quÚ separar desarrollo y producci¼n.
2.Puedo combinar archivos Compose con -f compose.yml -f
compose.prod.yml.
3.Puedo limpiar recursos huÓrfanos sin borrar datos por accidente.
4.Puedo diagnosticar fallas con estado, logs, inspect, exec y stats.
5.Puedo destruir y reconstruir el stack completo desde cero.
6.Puedo validar que Flask, PostgreSQL y Nginx funcionen como
proyecto integrado.
```

👤 **Guión del Docente (Lo que debes decir a los alumnos):**

"Estimados estudiantes, en la **Diapositiva 23** abordamos un punto fundamental: *CHECKLIST DE APRENDIZAJE ù SESIËN 6*. Resalten que en la vida real una aplicación web necesita varios componentes: app web, base de datos PostgreSQL, proxy Nginx. Docker Compose nos permite describir y levantar todo el stack con un único archivo `docker-compose.yml`."

 **Acción en Consola / Pizarra:** *Escribir en la pizarra o proyectar el diagrama del flujo correspondiente.*

 **Tip de Gestión del Aula:**

- *Hacer una pregunta de pausa activa: "¿Alguien tiene alguna duda sobre CHECKLIST DE APRENDIZAJE ò SESIÒN 6?"*

 Diapositiva 24: RESUMEN DE LA SESIÒN 6

Contenido de la PPT:

RESUMEN DE LA SESIÒN 6

- 1.La sesiòn 6 es mayormente pràctica e integradora; la teoríã se limita a dev/prod, limpieza y debugging.
 - 2.Multi-entorno permite reutilizar una base Compose y aplicar cambios por contexto.
 - 3.La limpieza debe distinguir entre recursos descartables y volúmenes con datos.
 - 4.Debugging profesional sigue un orden: estado, logs, inspect, exec y recursos.
 - 5.El proyecto final debe reconstruirse desde cero con comandos reproducibles.
- Aplicaciòn Flask + PostgreSQL + Nginx containerizada, con entornos dev/prod y despliegue reproducible.
- Resultado del curso

 **Guiòn del Docente (Lo que debes decir a los alumnos):**

"Estimados estudiantes, en la **Diapositiva 24** abordamos un punto fundamental: *RESUMEN DE LA SESIÒN 6*. Resalten que en la vida real una aplicaciòn web necesita varios componentes: app web, base de datos PostgreSQL, proxy Nginx. Docker Compose nos permite describir y levantar todo el stack con un único archivo `docker-compose.yml`."

 **Acción en Consola / Pizarra:** *Escribir en la pizarra o proyectar el diagrama del flujo correspondiente.*

 **Tip de Gestión del Aula:**

- *Hacer una pregunta de pausa activa: "¿Alguien tiene alguna duda sobre RESUMEN DE LA SESIÒN 6?"*

 Diapositiva 25: RESUMEN FINAL DEL PROGRAMA

Contenido de la PPT:

RESUMEN FINAL DEL PROGRAMA

Docker resuelve la reproducibilidad entre entornos.

Imàgenes y contenedores son la base del flujo.

Dockerfile permite construir imàgenes

propias.
 Compose levanta stacks multi-contenedor.
 Redes y volúmenes ordenan comunicación y datos.
 Aplicación completa: Flask, PostgreSQL y Nginx containerizados, listos para dev/prod y levantables en minutos.
 Resultado final
 Nginx actúa como punto de entrada del stack.
 Multi-stage reduce tamaño y superficie de ataque.
 Variables y archivos .env separan configuración.
 Logs, inspect, exec y stats ayudan a depurar.
 Dev/prod se separan con archivos Compose por entorno.

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 25** abordamos un punto fundamental: *RESUMEN FINAL DEL PROGRAMA*. Expliquen que el Dockerfile es la receta de cocina. Cada instrucción (**FROM**, **WORKDIR**, **COPY**, **RUN**, **CMD**) construye una capa estandarizada e inmutable sobre la cual se monta nuestra aplicación."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 25
./deploy.sh prod
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre RESUMEN FINAL DEL PROGRAMA?"

Diapositiva 26: CRISTIAN JAMPIER CHILENO SEGUNDO - OTI

Contenido de la PPT:

CRISTIAN JAMPIER CHILENO SEGUNDO - OTI
 ¡Muchas Gracias!
 Preguntas y Respuestas
 DOCKER
 Docente: Cristian Jampier Chileno Segundo
 Programa de Iniciación Tecnológica (PIT) 2026

Guión del Docente (Lo que debes decir a los alumnos):

"Estimados estudiantes, en la **Diapositiva 26** abordamos un punto fundamental: *CRISTIAN JAMPIER CHILENO SEGUNDO - OTI*. Revisen los conceptos clave presentados en pantalla y asegúrense de que los estudiantes comprendan cómo se conecta este punto con el laboratorio práctico de la sesión."

Acción en Consola / Pizarra:

```
# Demostración práctica recomendada para la Diapositiva 26
./deploy.sh prod
```

Tip de Gestión del Aula:

- *Hacer una pregunta de pausa activa:* "¿Alguien tiene alguna duda sobre CRISTIAN JAMPIER CHILENO SEGUNDO - OTI?"
-